

PROJECT REPORT

STUDENT RECORD MANAGEMENT SYSTEM (SRMS)



DONE BY

NAME: Sivalanka Lava Raju

REGISTRATION NUMBER: AP24110010861

CSE 2ND YEAR THIRD SEMESTER

SECTION AB

SUBMITTED TO

RAKESH RAMA RAJU

CODING SKILLS – I (CSE 201)

CERTIFICATE

This is to certify that the project entitled ***STUDENT RECORD MANAGEMENT SYSTEM*** has been successfully completed by **Sivalanka Lava Raju (AP24110010861)** as a required academic component of the course ***CSE 201: CODING SKILLS I*** during Semester 3 of Academic Year 2025-26 in the Department of Computer Science and Engineering at SRM University, AP. This work was executed under the guidance of the undersigned and is deemed to have successfully met all course requirements as of **06 / 12 / 2025**.

Rakesh Rama Raju P,
Campus Corporate Connect Trainer
Department of Computer Science and Engineering
SRM University, AP

ABSTRACT

This project presents a Student Record Management System developed in the C programming language to offer a reliable, secure and easy-to-use solution for managing student information. The system stores key details such as roll number, name and marks in text files, ensuring that all data is safely preserved even after the program closes. It provides the essential operations required for effective record management, including adding new entries, viewing stored data, searching for specific students, updating information and deleting records when necessary.

A role-based login system forms the foundation of its security and usability. Users can log in as Admin, Staff or Guest, and each role is granted only the level of access needed for its responsibilities. Admin users can perform all operations, staff users can view and search records and guest users are limited to viewing information. This structured approach not only protects the system from unauthorized modifications but also makes the interface simple and user-friendly for every type of user.

Overall, this project demonstrates the practical importance of core C programming concepts while showing how a straightforward application can effectively represent real-world data management. It stands as a highly valuable learning tool that combines simplicity, security and smooth usability to create a dependable student information management system.

INTRODUCTION

The **Student Record Management System** is a C-based application developed to efficiently manage and maintain student information using basic file-handling techniques. The system stores essential details such as roll number, name, and marks in external text files, ensuring that the data remains preserved even after the program is terminated. To ensure secure and organized access, the application incorporates a role-based login mechanism that restricts operations based on user privileges.

The system supports three distinct user roles: **Admin, Staff, and Guest**. Admin users possess complete access and can add, update, delete, search, and view student records. Staff members are granted permissions to search and view existing records, whereas Guest users are limited to viewing the data. This layered permission structure promotes data integrity and prevents unauthorized modifications.

In addition to managing student data, the project demonstrates the practical application of key concepts in the C programming language, including structures, file operations, string handling, and conditional logic. Overall, the Student Management System serves as a foundational example of how real-world data management applications can be designed and implemented using fundamental programming principles.

OBJECTIVES

The main objectives of this project are:

- To create an easy and efficient system for managing student information using C programming.
- To add a secure login system with different roles for Admin, Staff and Guest users.
- To give each user type the right level of access so the system is used properly.
- To store student details in text files so the data can be saved for a long time and accessed easily.
- To allow Admin users to add, view, search, update and delete student records.
- To allow Staff users to only view and search student information without making any changes.
- To give Guest users read-only access to keep the data safe.
- To show the practical use of file handling, structures and conditional statements in C.
- To protect the accuracy of the data by preventing unauthorized changes.
- To provide a simple, user-friendly, menu-based interface for smooth and easy navigation.

MODULES

The system consists of the following major modules:

1. Login Module

- Takes username and password.
- Checks credentials from credentials.txt.
- Identifies role: Admin, Staff, or Guest.

2. Role-Based Menu Module

- Shows menu according to user role.
- Controls access to operations for Admin, Staff, and Guest.

3. Student Addition Module (Admin Only)

- Allows adding new student details.
- Saves roll number, name, and marks in students.txt.

4. Display Students Module

- Reads all records from students.txt.
- Displays roll number, name, and marks.

5. Search Student Module

- Searches for a student by roll number.
- Shows matching record.

6. Update Student Module

- Updates student name or marks.
- Uses a temporary file to save updated data.

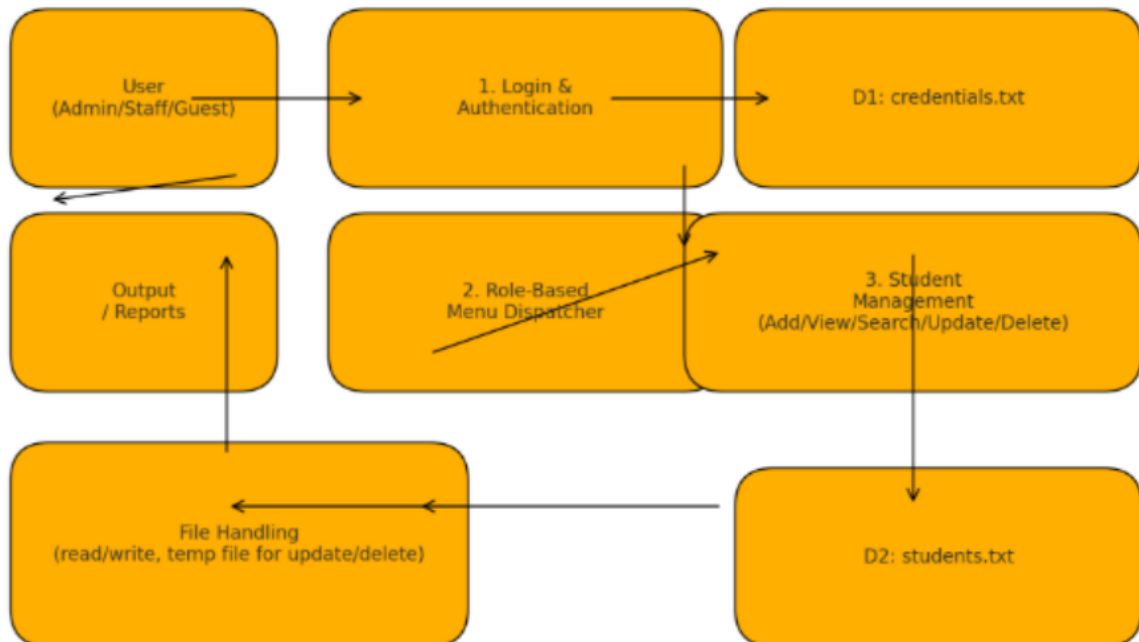
7. Delete Student Module (Admin Only)

- Deletes a student record using roll number.
- Rewrites file without the deleted entry.

8. File Handling Module

- Handles reading and writing of *credentials.txt* and *students.txt*.

DATA FLOW DIAGRAM



Explanation

1. Login & Authentication:
The system checks the entered username and password with *credentials.txt* and identifies the user role.
2. Role-Based Menu:
Based on the role (Admin, Staff, Guest), the appropriate menu is shown and the chosen option is passed to the next process.
3. Student Management:
Handles adding, viewing, searching, updating and deleting student records. Updates and deletions use a temporary file for safe rewriting.
4. File Handling:
All student data is stored in *students.txt*, ensuring permanent storage and reliable read/write operations.

CODE & OUTPUTS

➤ students.txt

```
output > students.txt
1 19822 Suriya 88.00
2 1001 Rahul 82.00
3
```

➤ Credentials.txt

```
output > credentials.txt
1 admin admin123 ADMIN
2 staff1 staff STAFF
3 guest1 guest GUEST
4 aadith713 aadith2006 ADMIN
```


➤ LOGIN & ADMIN MENU

```
void adminMenu() {
    int choice;
    while (1) {
        printf("\n==== ADMIN MENU =====\n");
        printf("1. Add Student\n");
        printf("2. Display Students\n");
        printf("3. Search Student\n");
        printf("4. Update Student\n");
        printf("5. Delete Student\n");
        printf("6. Logout\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: addStudent(); break;
            case 2: displayStudents(); break;
            case 3: searchStudent(); break;
            case 4: updateStudent(); break;
            case 5: deleteStudent(); break;
            case 6: printf("Logging out...\n"); return;
            default: printf("Invalid choice\n");
        }
    }
}
```

```
=====Login Screen=====
Username: admin
Password: admin123

==== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: █
```

After login, the Admin Menu appears, allowing the admin to add, view, search, update, or delete student records.

➤ ADD STUDENT

```
// ADD STUDENT
void addStudent() {
    FILE *fp = fopen(STUDENT_FILE, "a");
    struct Student st;

    if (!fp) {
        printf("Error opening file\n");
        return;
    }

    printf("Enter Roll No: ");
    scanf("%d", &st.roll);

    printf("Enter Name: ");
    scanf("%s", st.name);
    printf("Enter Marks: ");
    scanf("%f", &st.marks);

    fprintf(fp, "%d %s %.2f\n", st.roll, st.name, st.marks);
    fclose(fp);

    printf("Student Added Successfully!\n");
}
```

```
==== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 1
Enter Roll No: 1001
Enter Name: Rahul
Enter Marks: 82
Student Added Successfully!
```

This screen allows the admin to add a new student record by entering the roll number, name, and marks. After submission, the record is successfully stored in the system.

➤ DISPLAY STUDENT

```
174 // DISPLAY STUDENTS
175 void displayStudents() {
176     FILE *fp = fopen(STUDENT_FILE, "r");
177     struct Student st;
178
179     if (!fp) {
180         printf("No Student records found\n");
181         return;
182     }
183
184     printf("\nRoll\tName\tMarks\n");
185     printf("-----\n");
186
187     while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) == 3) {
188         printf("%d\t%s\t%.2f\n", st.roll, st.name, st.marks);
189     }
190     fclose(fp);
191 }
192
```

```
==== ADMIN MENU ====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 2
```

Roll	Name	Marks
19822	Suriya	88.00
1001	Rahul	82.00
10713	Aadith	82.00

This output shows the list of all stored student records. It displays roll numbers, names, and marks in a structured table format.

➤ SEARCH STUDENT

```
193 // SEARCH STUDENT
194 void searchStudent() {
195     FILE *fp = fopen(STUDENT_FILE, "r");
196     struct Student st;
197     int roll, found = 0;
198
199     if (!fp) {
200         printf("Error opening file\n");
201         return;
202     }
203
204     printf("Enter Roll to search: ");
205     scanf("%d", &roll);
206
207     while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) == 3) {
208         if (st.roll == roll) {
209             printf("Record found\n");
210             printf("Roll: %d\nName: %s\nMarks: %.2f\n", st.roll, st.name, st.marks);
211             found = 1;
212             break;
213         }
214     }
215
216     if (!found)
217         printf("Record not found\n");
218
219     fclose(fp);
220 }
```

```
==== ADMIN MENU ====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 3
Enter Roll to search: 10713
Record found
Roll: 10713
Name: Aadith
Marks: 82.00
```

This screen lets the user search for a student using the roll number. If the record exists, the student's details are displayed.

➤ UPDATE STUDENT

```
222 // UPDATE STUDENT
223 void updateStudent() {
224     FILE *fp = fopen(STUDENT_FILE, "r");
225     FILE *temp = fopen("temp.txt", "w");
226     struct Student st;
227     int roll, found = 0;
228
229     if (!fp || !temp) {
230         printf("Error opening file\n");
231         return;
232     }
233
234     printf("Enter Roll to update: ");
235     scanf("%d", &roll);
236
237     while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) == 3) {
238         if (st.roll == roll) {
239             printf("Enter new Name: ");
240             scanf("%s", st.name);
241             printf("Enter new Marks: ");
242             scanf("%f", &st.marks);
243             found = 1;
244         }
245         fprintf(temp, "%d %s %.2f\n", st.roll, st.name, st.marks);
246     }
247
248     fclose(fp);
249     fclose(temp);
250
251     remove(STUDENT_FILE);
252     rename("temp.txt", STUDENT_FILE);
253
254     if (found)
255         printf("Record Updated Successfully!\n");
256     else
257         printf("Record not found\n");
258 }
259
```

```
==== ADMIN MENU ====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 4
Enter Roll to update: 10713
Enter new Name: Vecna
Enter new Marks: 99
Record Updated Successfully!
```

Roll	Name	Marks
19822	Suriya	88.00
1001	Rahul	82.00
10713	Vecna	99.00

The update screen enables modification of an existing student's details. The admin can update the student's name and marks after entering the roll number.

➤ DELETE STUDENT

```
// DELETE STUDENT
void deleteStudent() {
    FILE *fp = fopen(STUDENT_FILE, "r");
    FILE *temp = fopen("temp.txt", "w");
    struct Student st;
    int roll, found = 0;

    if (!fp || !temp) {
        printf("Error opening file\n");
        return;
    }

    printf("Enter Roll to delete: ");
    scanf("%d", &roll);

    while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) == 3) {
        if (st.roll == roll) {
            found = 1;
            continue;
        }
        fprintf(temp, "%d %s %.2f\n", st.roll, st.name, st.marks);
    }

    fclose(fp);
    fclose(temp);

    remove(STUDENT_FILE);
    rename("temp.txt", STUDENT_FILE);

    if (found)
        printf("Record Deleted Successfully!\n");
    else
        printf("Record not found\n");
}
```

==== ADMIN MENU =====

1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout

Enter choice: 5

Enter Roll to delete: 10713

Record Deleted Successfully!

Enter choice: 2

Roll	Name	Marks
19822	Suriya	88.00
1001	Rahul	82.00

This output shows the result of deleting a student record. The system removes the matching roll number from the database and confirms the action.

CONCLUSION

The Student Record Management System is a simple program that helps store and manage student information in an easy and organised way. Users can add, view, search, update, and delete student records depending on their role. Admin, Staff, and Guest users have different levels of access, which helps keep the records safe and prevents unwanted changes. The system is menu-driven and easy to understand, making it simple for anyone to use.

This project shows how basic C programming concepts like structures, functions, loops, conditions, and simple file storage can be used to build a small working application. It performs all the main tasks needed for handling student records in a clear and reliable way. Overall, the Student Record Management System is a good example of how useful applications can be created using simple and fundamental C programming techniques.